C++异常处理机制分析

C++异常处理机制分析

在C++编程中，异常处理是我们构建健壮软件的关键环节。

梳理记录一下，C++的 `try-catch` 异常捕获机制，分析一下其工作原理。笔记记录。。

一、异常处理基础

1.1 C++异常处理的核心组件

C++异常处理基于三个关键字：

- `try`：包围可能抛出异常的代码块
- `catch`：捕获并处理特定类型的异常
- `throw`：抛出一个异常对象

基本语法如下：

```
try {
    // 可能抛出异常的代码
    if (condition_not_met) {
        throw SomeException("错误信息");
```

```
    }
} catch (const SomeException& e) {
    // 处理SomeException类型的异常
    std::cerr << "捕获到异常: " << e.what() << std::endl;
} catch (...) {
    // 捕获任何其他异常
    std::cerr << "捕获到未知异常" << std::endl;
}
```

1.2 异常处理流程

当异常被抛出时，C++运行时系统会：

- **暂停当前函数执行**：立即停止当前try块内的代码执行
- **查找匹配的catch处理器**：沿着调用栈向上查找能够处理该异常的catch块
- **栈解退 (Stack Unwinding)**：销毁try块中创建的所有自动变量
- **执行catch块**：如果找到匹配的catch块，执行其中的代码
- **恢复执行**：从catch块之后的语句继续执行

如果找不到匹配的catch块，程序将调用 `std::terminate()` 终止执行。

`std::terminate()` 是C++标准库中的一个函数，它的作用是终止程序的执行。默认情况下，它会调用std::abort()，这会导致程序异常终止，不执行任何清理操作。

简单来说：如果异常被抛出却没被捕获，程序会崩溃。

二、异常处理的深层机制

2.1 栈解退机制

栈解退是异常处理的核心机制，它确保资源在异常传播过程中被正确释放：

当异常发生时，C++会自动按照创建顺序的逆序，依次调用作用域内所有对象的析构函数，以确保即使程序执行路径被异常中断，资源（如内存、文件句柄等）也不会泄漏。

举个例子来看，基本栈解退过程

```
#include <iostream>
#include <stdexcept>

class Resource {
    std::string name_;
public:
    Resource(const std::string& name) : name_(name) {
        std::cout << "Resource " << name_ << " 创建" << std::endl;
    }

    ~Resource() {
        std::cout << "Resource " << name_ << " 销毁" << std::endl;
    }
};

void function_c() {
    Resource r3("C");
    std::cout << "即将在function_c中抛出异常..." << std::endl;
    throw std::runtime_error("故意抛出的异常");
    // r3在此自动销毁（栈解退）
    std::cout << "这行代码永远不会执行" << std::endl;
```

```
}

void function_b() {
    Resource r2("B");
    std::cout << "进入function_b，调用function_c..." << std::endl;
    function_c();
    // r2在此自动销毁（栈解退）
    std::cout << "这行代码也不会执行" << std::endl;
}

void function_a() {
    Resource r1("A");
    try {
        std::cout << "进入function_a，调用function_b..." << std::endl;
        function_b();
    } catch (const std::exception& e) {
        std::cout << "异常在function_a中被捕获: " << e.what() << std::endl;
    }
    // r1正常销毁（函数结束）
    std::cout << "function_a结束" << std::endl;
}

int main() {
    std::cout << "开始执行..." << std::endl;
    function_a();
    std::cout << "程序正常结束" << std::endl;
    return 0;
}
```

编译运行观察

```
$ g++ -o su StackUnwinding.cc -std=c++11
$ ./su
开始执行...
Resource A 创建
进入function_a，调用function_b...
Resource B 创建
进入function_b，调用function_c...
Resource C 创建
即将在function_c中抛出异常...
Resource C 销毁
Resource B 销毁
异常在function_a中被捕获：故意抛出的异常
function_a结束
Resource A 销毁
程序正常结束
```

知乎 @大橘为重

编译运行

如上，在抛出异常时：

资源C和B都被正确销毁，尽管它们所在的函数执行被异常中断

资源销毁顺序是创建顺序的逆序：先C，后B，

最后A function_c()和function_b()中抛出异常后的代码没有执行

这是C++确保资源安全释放的重要机制，也是**RAII**[+]**（资源获取即初始化）**模式的基础。

2.2 栈解退与RAII设计模式

栈解退机制的强大之处还在于它与RAII设计模式可以完美配合。

RAII原则：

- 在构造函数中获取资源

- 在析构函数中释放资源

- 通过对象生命周期自动管理资源

为什么栈解退被称为异常处理的"核心机制"？

因为它解决了异常处理中最关键的问题：在非正常执行路径下的资源管理。

它确保：

- 所有局部资源都被释放，无论执行路径如何

- 无需额外代码：不需要显式的资源清理代码，析构函数自动调用

- 代码简洁：避免了大量的try/catch/finally嵌套

- 异常安全：即使在复杂的调用链和多次异常情况下，也能按照确定的顺序（创建顺序的逆序）释放资源，处理依赖关系，资源也能被正确释放

- 确定性：资源释放的顺序和时机完全可预测

通过这种机制，它从根本上解决了异常情况下的资源管理问题，这在任何错误处理系统中都是最基础也是最关键的部分。

2.3 栈解退的技术细节

栈解退通常通过以下技术实现：

（1）异常表：编译器生成描述哪些代码块需要清理的表

异常表是编译器生成的特殊数据结构，包含了程序中每个可能抛出异常的函数的信息，

主要包括：

保护区域（Protected Regions）：标识代码中可能抛出异常的区域

处理区域（Handler Regions）：与每个保护区域关联的异常处理代码

清理操作（Cleanup Actions）：需要在异常传播时执行的操作

```
void function() {
    Resource r;  // 需要析构的对象
    try {
        might_throw();
    } catch(const std::exception& e) {
        handle_exception(e);
    }
}
```

编译器会生成异常表，大致包含：

try块的边界（字节码偏移量）

catch块的位置和它处理的异常类型（std::exception）

Resource对象需要在异常传播期间析构的信息

（2）栈帧遍历：运行时系统从当前栈帧开始，向调用者的方向遍历

当异常被抛出时：

保存异常对象：创建异常对象的副本并在堆上分配空间

查找当前函数的异常表：检查是否有匹配的处理器

如果没有处理器，开始解退过程：

- 从当前栈帧获取返回地址
- 确定调用者的栈帧位置
- 执行当前栈帧的清理代码（调用析构函数）
- 恢复调用者的寄存器状态
- 移动到调用者的栈帧
- 重复上述过程直到找到处理器或达到main函数

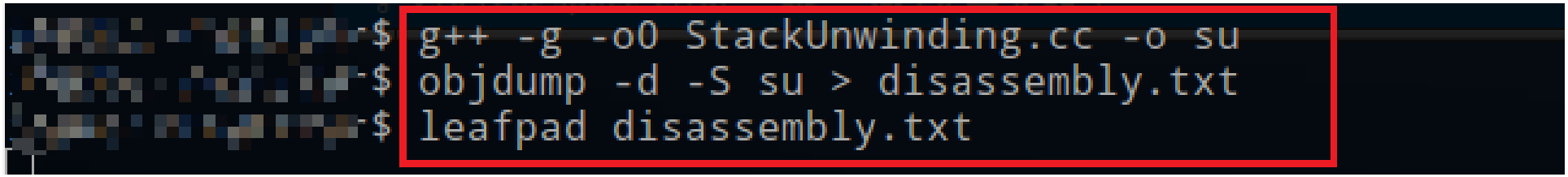
（3）析构函数调用：针对每个栈帧中的自动变量调用析构函数

可以使用objdump反汇编查看GCC编译生成的异常表

```
g++ -g -o0 StackUnwinding.cc -o su
# 参数说明：
# -g：生成调试信息
# -O0：禁用优化，使生成的代码更接近源代码结构

objdump -d -S su > disassembly.txt
# 参数说明：
# -d：反汇编可执行段
# -S：混合源代码（如果有调试信息）
# 这会产生大量输出，通常需要筛选或重定向到文件：这里重定向到disassembly.txt

readelf --debug-dump=frames su
# 对于更深入的解析，使用readelf可能更有帮助：
```



编译

```
$ readelf --debug-dump=frames su
Contents of the .eh_frame section:

00000000 00000014 00000000 CIE
  Version:               1
  Augmentation:          "zR"
  Code alignment factor: 1
  Data alignment factor: -4
  Return address column: 8
  Augmentation data:     1b

  DW_CFA_def_cfa: r4 (esp) ofs 4
  DW_CFA_offset: r8 (eip) at cfa-4
  DW_CFA_nop
  DW_CFA_nop

00000018 00000020 0000001c FDE cie=00000000 pc=080488d0..08048a50
  DW_CFA_def_cfa_offset: 8
  DW_CFA_advance_loc: 6 to 080488d6
  DW_CFA_def_cfa_offset: 12
  DW_CFA_advance_loc: 10 to 080488e0
  DW_CFA_def_cfa_expression (DW_OP_breg4 (esp): 4; DW_OP_
_OP_plus)
```

知乎 @大橘为重

分析异常

2.4 性能影响

异常处理机制通常遵循"零开销原则"：不使用时不付出代价，使用时性能影响最小，几乎没有额外开销。但在异常被抛出时，执行必要的清理工作，开销可能很大：

- **栈解退成本**：需要调用析构函数并释放资源
- **运行时开销**：查找匹配的异常处理器需要时间

2.5 异常规格与noexcept

C++11引入了 `noexcept` 说明符，用于声明函数不会抛出异常：

```
void safe_function() noexcept {
    // 该函数保证不抛出异常
}
```

当标记为 `noexcept` 的函数实际尝试抛出异常时：

- 异常被抛出，开始正常的栈解退过程
- 但当异常传播到函数边界时，运行时系统检测到这是一个 `noexcept` 函数
- 调用 `std::terminate()` 立即终止程序，不会继续将异常传播给调用者

noexcept适合场景：

- 析构函数：几乎所有析构函数都应标记为noexcept。一般析构函数默认就是noexcept的，通常不需要显式标记。析构函数抛出异常可能导致未定义行为，尤其是在栈解退过程中。
- 移动构造函数和移动赋值运算符：允许容器执行优化
- 内存管理和底层操作：如swap()、clear()等
- 无法恢复的操作：如果异常发生时无法采取恢复措施
- 性能关键代码：需要避免异常处理开销的部分

三、常用场景

3.1 异常类层次设计

良好的异常层次结构可以提高代码的可维护性和灵活性：

```
// 基础异常类
class ApplicationException : public std::exception {
protected:
    std::string message_;
public:
    explicit ApplicationException(const std::string& message) : message_(message) {}
    virtual const char* what() const noexcept override {
        return message_.c_str();
    }
};

// 专用异常类
// 数据库异常
class DatabaseException : public ApplicationException {
public:
    explicit DatabaseException(const std::string& message)
        : ApplicationException("Database error: " + message) {}
};

// 网络异常
class NetworkException : public ApplicationException {
public:
    explicit NetworkException(const std::string& message)
```

```
        : ApplicationException("Network error: " + message) {}
};
```

这种设计允许捕获特定类型的异常或更通用的异常：

```
try {
    // 可能抛出不同类型异常的代码
} catch (const DatabaseException& e) {
    // 处理数据库异常
} catch (const NetworkException& e) {
    // 处理网络异常
} catch (const ApplicationException& e) {
    // 处理其他应用程序异常
} catch (const std::exception& e) {
    // 处理标准库异常
} catch (...) {
    // 处理其他所有异常
}
```

按引用捕获异常是最佳实践：

- **避免对象切片**：保留派生类信息
- **提高效率**：避免不必要的对象复制
- **使用多态**：可以调用派生类的虚函数

3.2 异常的嵌套与重新抛出

有时需要捕获异常，执行一些操作，然后重新抛出：

```
try {
    // 低级操作
} catch (const std::exception& e) {
    // 记录错误
    logger.log("发生错误: " + std::string(e.what()));

    // 重新抛出相同的异常
    throw;

    // 或者包装并抛出新异常
    // throw HighLevelException(e.what());
}
```

使用不带参数的 `throw` 语句可以保留原始异常的类型和所有信息。

3.3 异常安全保证级别

通常定义三个级别的异常安全保证：

1. **基本保证**：异常发生时不会泄露资源，对象仍处于有效但可能不确定的状态
2. **强保证**：异常发生时，程序状态不变（要么完全成功，要么回到原始状态）
3. **不抛出保证**：操作保证不会抛出异常

```
// 强异常保证的例子
void update_data(std::vector<Data>& vec, const Data& new_item) {
    // 创建副本，修改副本
    std::vector<Data> temp = vec;
    temp.push_back(new_item);
```

```
    // 只有当所有操作成功时才替换原始数据
    // 如果中途发生异常，原始数据不变
    vec.swap(temp);
}
```

四、总结

C++的异常处理机制很强大，我们可以结合项目实际需求设计合理的异常处理策略，提高代码的可靠性。不过要结合具体的场景，并不是所有错误情况都适合使用异常处理，有的时候更适合用错误码。。。

- E.14: Use purpose-designed user-defined types as exceptions (not built-in types)
- E.15: Catch exceptions from a hierarchy by reference
- E.16: Destructors, deallocation, and `swap` must never fail
- E.17: Don't try to catch every exception in every function
- E.18: Minimize the use of explicit `try/catch`

知乎 @大橘为重

核心原则

五、参考资料

1. C++ Core Guidelines：Exception Handling
2. ISO C++ FAQ：Exceptions and Error Handling
3. Effective Modern C++
4. C++ Standard Library：<exception>，<stdexcept>
5. Microsoft Learn：Modern C++ best practices for exceptions and error handling

送礼物

还没有人送礼物，鼓励一下作者吧

发布于 2025-03-25 21:49 · 上海

C++ 异常处理

在C++异常处理中，栈解退（Stack Unwinding）过程中如果局部变量的析构函数再次抛出异常，会导致程序直接调用`terminate()`终止运行，栈解退过程会被强制中断。以下是关键机制分析：

双重异常的危险性

当栈解退过程中析构函数抛出第二个异常时，C++标准规定此时程序必须终止。因为系统无法同时处理两个活跃的异常，这会导致资源泄漏和未定义行为。

底层实现原理

栈解退依赖编译器生成的`UNWINDTBL`结构体数组，按逆序调用局部对象的析构函数。若某个析构函数抛出异常，该异常会覆盖当前正在处理的异常，触发`terminate()`。

典型场景示例

```
class Resource {
public:
```

```
    ~Resource() { throw std::runtime_error("析构异常"); } // 危险操作
};

void func() {
    Resource res;
    throw std::logic_error("主异常"); // 栈解退时触发析构异常
}
```

上述代码会导致程序立即终止。

最佳实践建议

析构函数应通过`try-catch`块捕获所有内部异常

使用RAII管理资源时，确保析构函数为`noexcept`

复杂清理操作应移至普通成员函数而非析构函数

总结：C++通过强制终止程序来保证异常处理系统的确定性，这是比资源泄漏更可控的失败方式。